

Experiment-02

Aim:

- A. Write an SQL solution to calculate the total number of bank accounts belonging to each salary category. The classification categories are:
- "Low Salary": All monthly salaries strictly less than \$20,000.
 - "Average Salary": All monthly salaries in the inclusive range [\$20,000, \$50,000].
 - "High Salary": All monthly salaries strictly greater than \$50,000. The final result table must report all three categories, even if a category contains zero accounts.
- B. Two legacy HR systems (A & B) have separate records of employee salaries. These records may overlap. Management wants to merge these datasets and identify each unique employee (by EmpID) along with their lowest recorded salary across both systems.
Objective:
- Combine two tables A and B
 - Return each EmpID with their lowest salary, and the corresponding Ename.
- C. A multinational organization maintains separate records for employee information and department details. The management wants to identify the employees who receive the highest salary within their respective departments for an annual performance report. If more than one employee earns the same highest salary in a department, all such employees should be included in the report. The final output should display the **Department Name**, **Employee Name**, and **Salary**, arranged in ascending order of department name.
- D. Find the 2nd highest earning employee from the same schema.

Tools / Platform Used:

- Windows
- PostgreSQL
- PgAdmin

Theory:

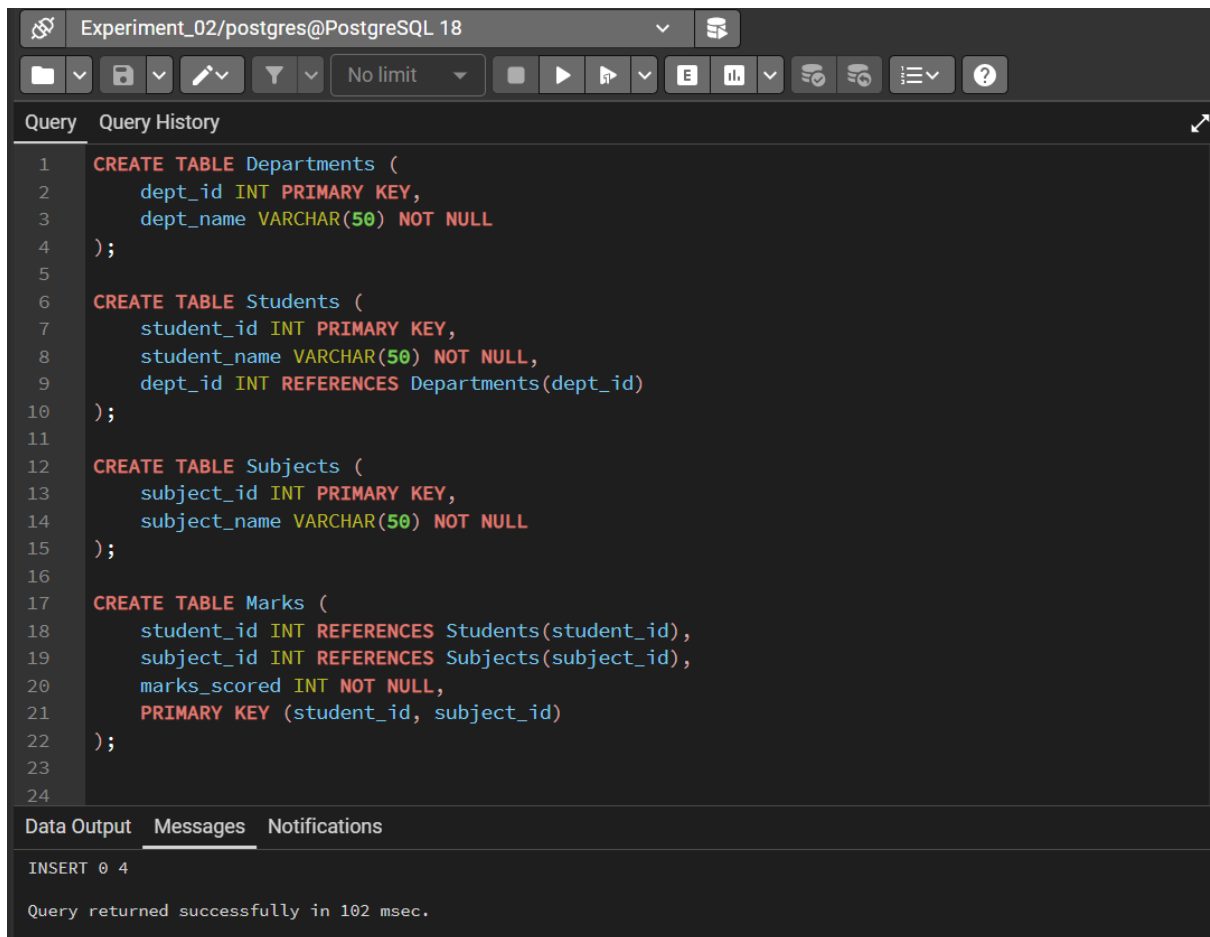
A subquery is a SQL query nested inside a larger statement, such as a SELECT, INSERT, UPDATE, or DELETE statement. Also known as an inner query or nested query, it acts as a helper to pass dynamically retrieved data to the primary outer query.

Types of Subqueries are:

- Scalar Subquery – returns exactly one value for the output
- Multi-row Subquery – returns multiple rows but only one column
- Multi-column – returns multiple columns and rows
- Correlated – subquery which is linked to the outer query

Code and Output:

A.



```
Experiment_02/postgres@PostgreSQL 18
No limit
Query History
1 CREATE TABLE Departments (
2     dept_id INT PRIMARY KEY,
3     dept_name VARCHAR(50) NOT NULL
4 );
5
6 CREATE TABLE Students (
7     student_id INT PRIMARY KEY,
8     student_name VARCHAR(50) NOT NULL,
9     dept_id INT REFERENCES Departments(dept_id)
10 );
11
12 CREATE TABLE Subjects (
13     subject_id INT PRIMARY KEY,
14     subject_name VARCHAR(50) NOT NULL
15 );
16
17 CREATE TABLE Marks (
18     student_id INT REFERENCES Students(student_id),
19     subject_id INT REFERENCES Subjects(subject_id),
20     marks_scored INT NOT NULL,
21     PRIMARY KEY (student_id, subject_id)
22 );
23
24
Data Output Messages Notifications
INSERT 0 4
Query returned successfully in 102 msec.
```

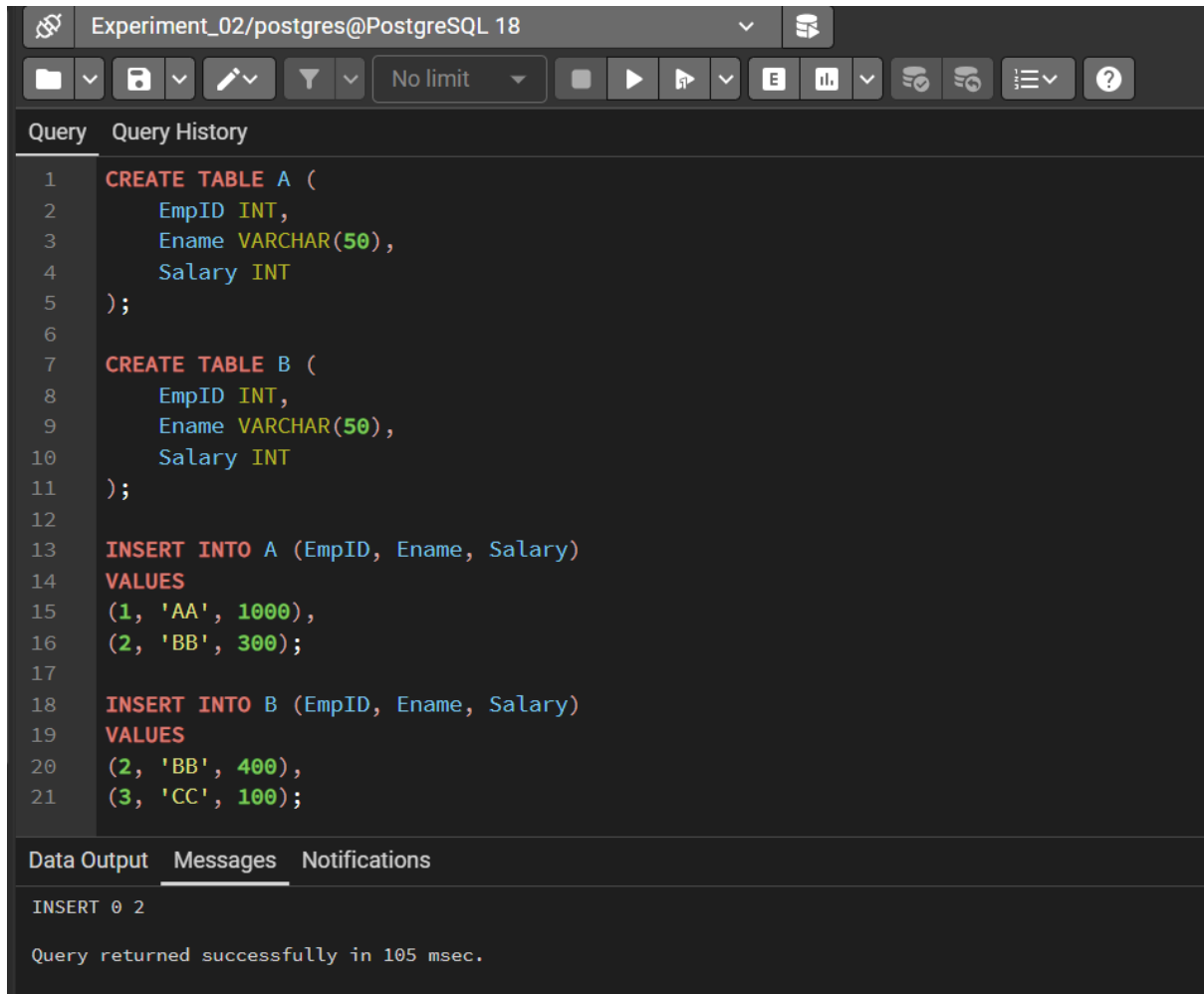
```
Experiment_02/postgres@PostgreSQL 18
No limit
Query Query History
19 subject_id INT REFERENCES Subjects(subject_id),
20 marks_scored INT NOT NULL,
21 PRIMARY KEY (student_id, subject_id)
22 );
23
24
25 INSERT INTO Departments (dept_id, dept_name) VALUES
26 (1, 'Computer Science'),
27 (2, 'Mathematics');
28
29 INSERT INTO Students (student_id, student_name, dept_id) VALUES |
30 (101, 'Alex', 1),
31 (102, 'Bella', 1),
32 (103, 'Charlie', 2);
33
34 INSERT INTO Subjects (subject_id, subject_name) VALUES
35 (501, 'Data Structures'),
36 (502, 'Calculus');
37
38 INSERT INTO Marks (student_id, subject_id, marks_scored) VALUES
39 (101, 501, 85),
40 (101, 502, 90),
41 (102, 501, 75),
42 (103, 502, 95);
Data Output Messages Notifications
INSERT 0 4
Query returned successfully in 102 msec.
```

```
Experiment_02/postgres@PostgreSQL 18
No limit
Query Query History
1 SELECT
2     D.dept_name AS DEPARTMENT,
3     AVG(M.marks_scored) AS "AVERAGE MARKS"
4 FROM Departments AS D
5 JOIN
6 Students AS S
7 ON
8 D.dept_id = S.dept_id
9 JOIN Marks as M
10 ON
11 M.Student_id = S.Student_id
12 GROUP BY DEPARTMENT
13 ORDER BY "AVERAGE MARKS" DESC;
Data Output Messages Notifications
Showing rows: 1 to 2


|   | department<br>character varying (50) | AVERAGE MARKS<br>numeric |
|---|--------------------------------------|--------------------------|
| 1 | Mathematics                          | 95.0000000000000000      |
| 2 | Computer Science                     | 83.3333333333333333      |


```

B.

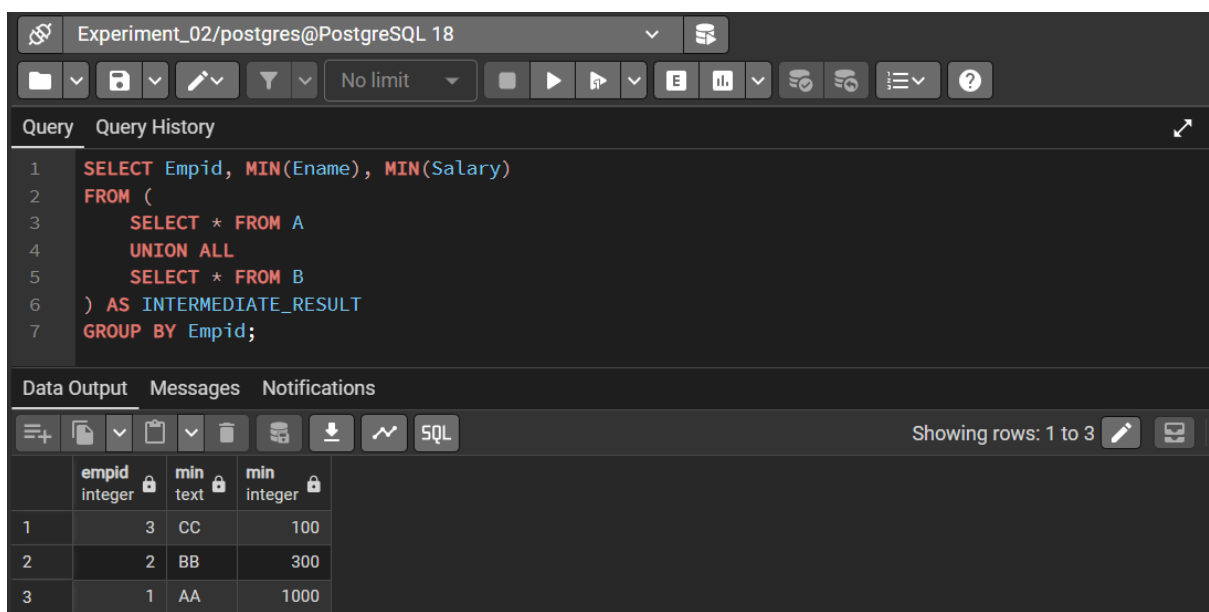


The screenshot shows a PostgreSQL query editor interface. The title bar indicates the connection is 'Experiment_02/postgres@PostgreSQL 18'. The query editor contains the following SQL code:

```
1 CREATE TABLE A (  
2     EmpID INT,  
3     Ename VARCHAR(50),  
4     Salary INT  
5 );  
6  
7 CREATE TABLE B (  
8     EmpID INT,  
9     Ename VARCHAR(50),  
10    Salary INT  
11 );  
12  
13 INSERT INTO A (EmpID, Ename, Salary)  
14 VALUES  
15 (1, 'AA', 1000),  
16 (2, 'BB', 300);  
17  
18 INSERT INTO B (EmpID, Ename, Salary)  
19 VALUES  
20 (2, 'BB', 400),  
21 (3, 'CC', 100);
```

Below the query editor, the 'Data Output' tab is selected, showing the result of the last query:

```
INSERT 0 2  
  
Query returned successfully in 105 msec.
```



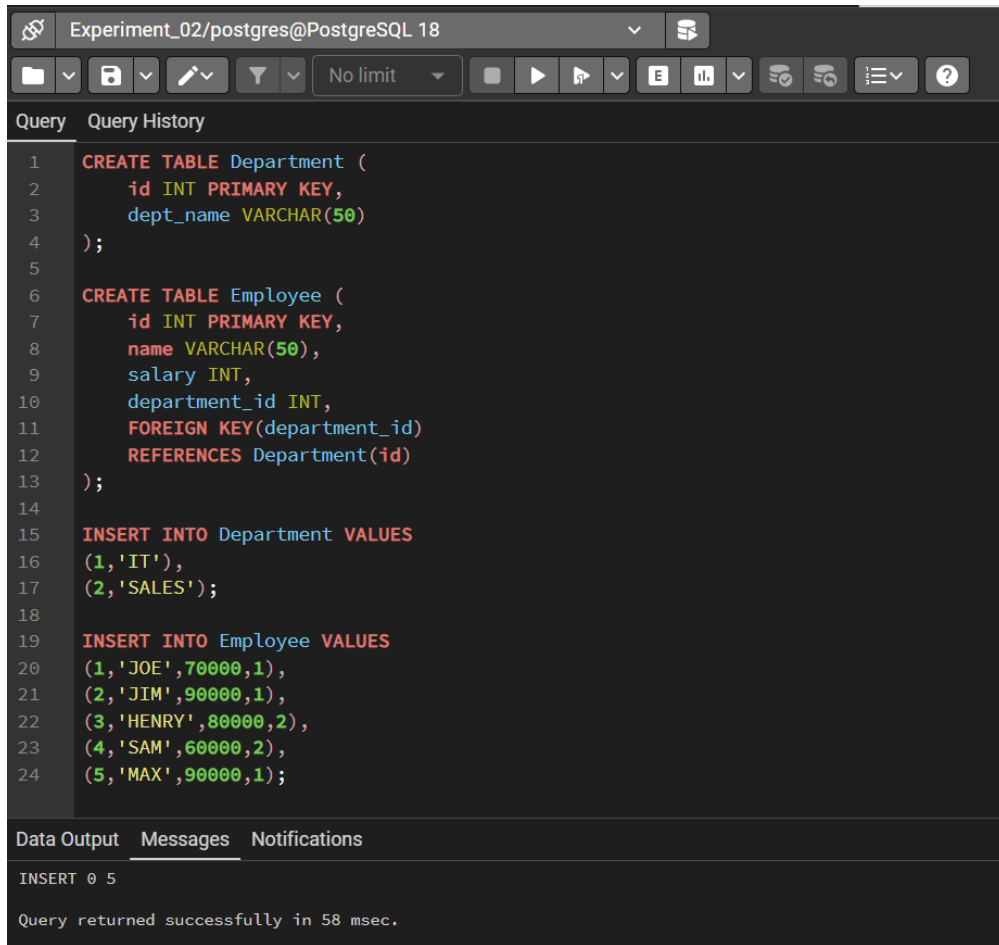
The screenshot shows the same PostgreSQL query editor interface. The query editor contains the following SQL code:

```
1 SELECT Empid, MIN(Ename), MIN(Salary)  
2 FROM (  
3     SELECT * FROM A  
4     UNION ALL  
5     SELECT * FROM B  
6 ) AS INTERMEDIATE_RESULT  
7 GROUP BY Empid;
```

Below the query editor, the 'Data Output' tab is selected, showing the result of the query. The interface includes a toolbar with icons for various actions and a status bar indicating 'Showing rows: 1 to 3'.

	empid integer	min text	min integer
1	3	CC	100
2	2	BB	300
3	1	AA	1000

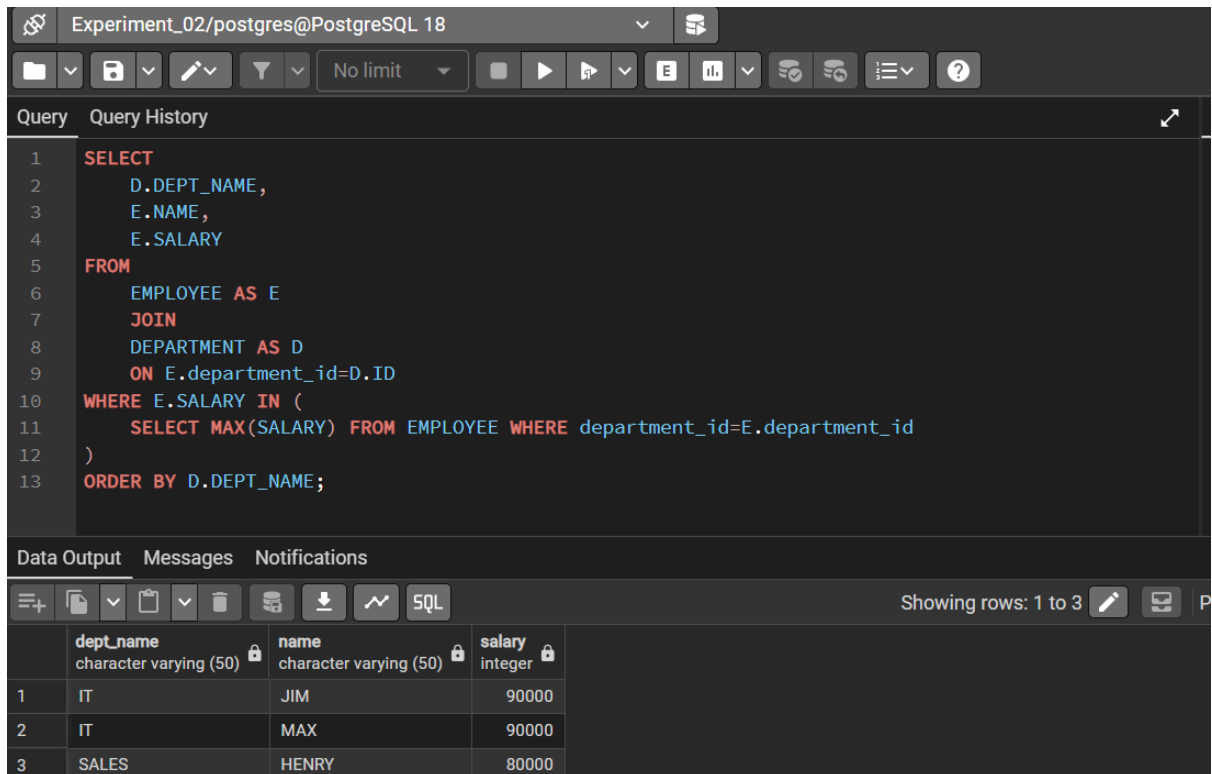
C.



The screenshot shows a PostgreSQL query editor with the following SQL code:

```
1 CREATE TABLE Department (  
2     id INT PRIMARY KEY,  
3     dept_name VARCHAR(50)  
4 );  
5  
6 CREATE TABLE Employee (  
7     id INT PRIMARY KEY,  
8     name VARCHAR(50),  
9     salary INT,  
10    department_id INT,  
11    FOREIGN KEY(department_id)  
12    REFERENCES Department(id)  
13 );  
14  
15 INSERT INTO Department VALUES  
16 (1, 'IT'),  
17 (2, 'SALES');  
18  
19 INSERT INTO Employee VALUES  
20 (1, 'JOE', 70000, 1),  
21 (2, 'JIM', 90000, 1),  
22 (3, 'HENRY', 80000, 2),  
23 (4, 'SAM', 60000, 2),  
24 (5, 'MAX', 90000, 1);
```

The bottom panel shows the "Data Output" tab with the message: "INSERT 0 5" and "Query returned successfully in 58 msec."



The screenshot shows a PostgreSQL query editor with the following SQL code:

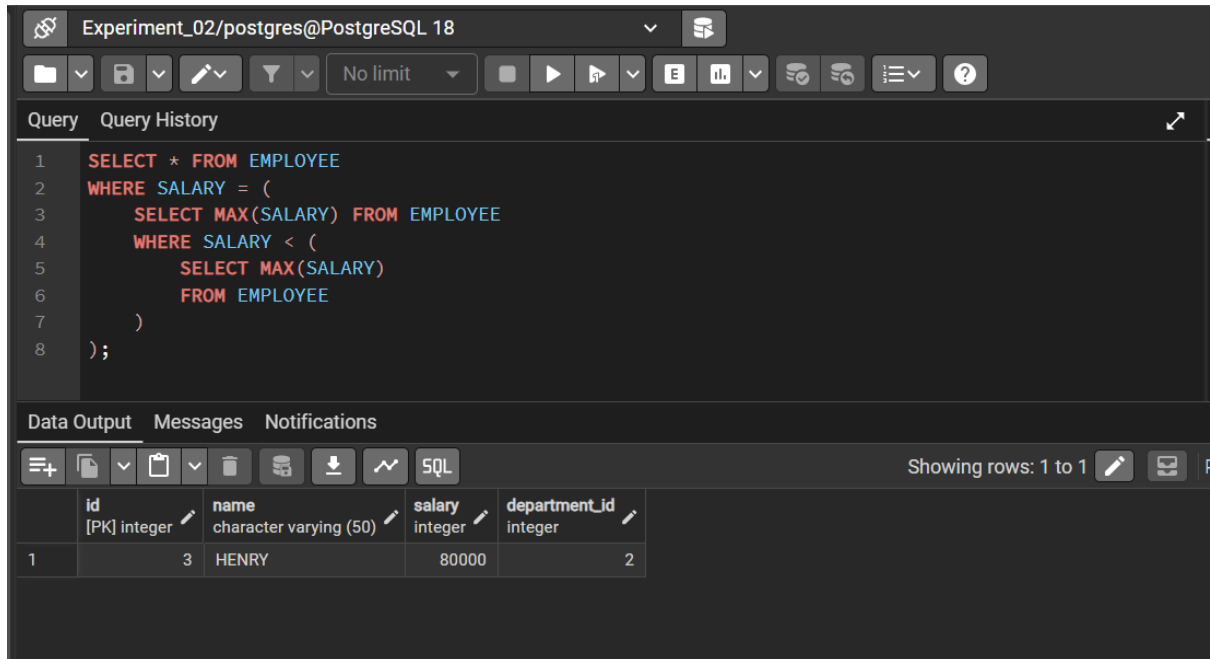
```
1 SELECT  
2     D.DEPT_NAME,  
3     E.NAME,  
4     E.SALARY  
5 FROM  
6     EMPLOYEE AS E  
7     JOIN  
8     DEPARTMENT AS D  
9     ON E.department_id=D.ID  
10 WHERE E.SALARY IN (  
11     SELECT MAX(SALARY) FROM EMPLOYEE WHERE department_id=E.department_id  
12 )  
13 ORDER BY D.DEPT_NAME;
```

The bottom panel shows the "Data Output" tab with the following table:

	dept_name character varying (50)	name character varying (50)	salary integer
1	IT	JIM	90000
2	IT	MAX	90000
3	SALES	HENRY	80000

The bottom right corner of the screenshot shows "Showing rows: 1 to 3" and a "P" button.

D.



The screenshot shows a PostgreSQL query editor interface. The top bar indicates the connection is 'Experiment_02/postgres@PostgreSQL 18'. The query editor contains the following SQL code:

```
1 SELECT * FROM EMPLOYEE
2 WHERE SALARY = (
3     SELECT MAX(SALARY) FROM EMPLOYEE
4     WHERE SALARY < (
5         SELECT MAX(SALARY)
6         FROM EMPLOYEE
7     )
8 );
```

Below the query editor, the 'Data Output' tab is active, showing a single row of results. The table has four columns: 'id' (integer, primary key), 'name' (character varying (50)), 'salary' (integer), and 'department_id' (integer). The row shows an employee with id 3, name HENRY, salary 80000, and department_id 2.

	id [PK] integer	name character varying (50)	salary integer	department_id integer
1	3	HENRY	80000	2

Result:

The queries for the given tasks were written and executed successfully. The tasks given were completed successfully.

Learning Outcomes:

- Practiced PostgreSQL
- Got hands-on experience on working with subqueries in PostgreSQL.
- Understood the concept of subqueries.